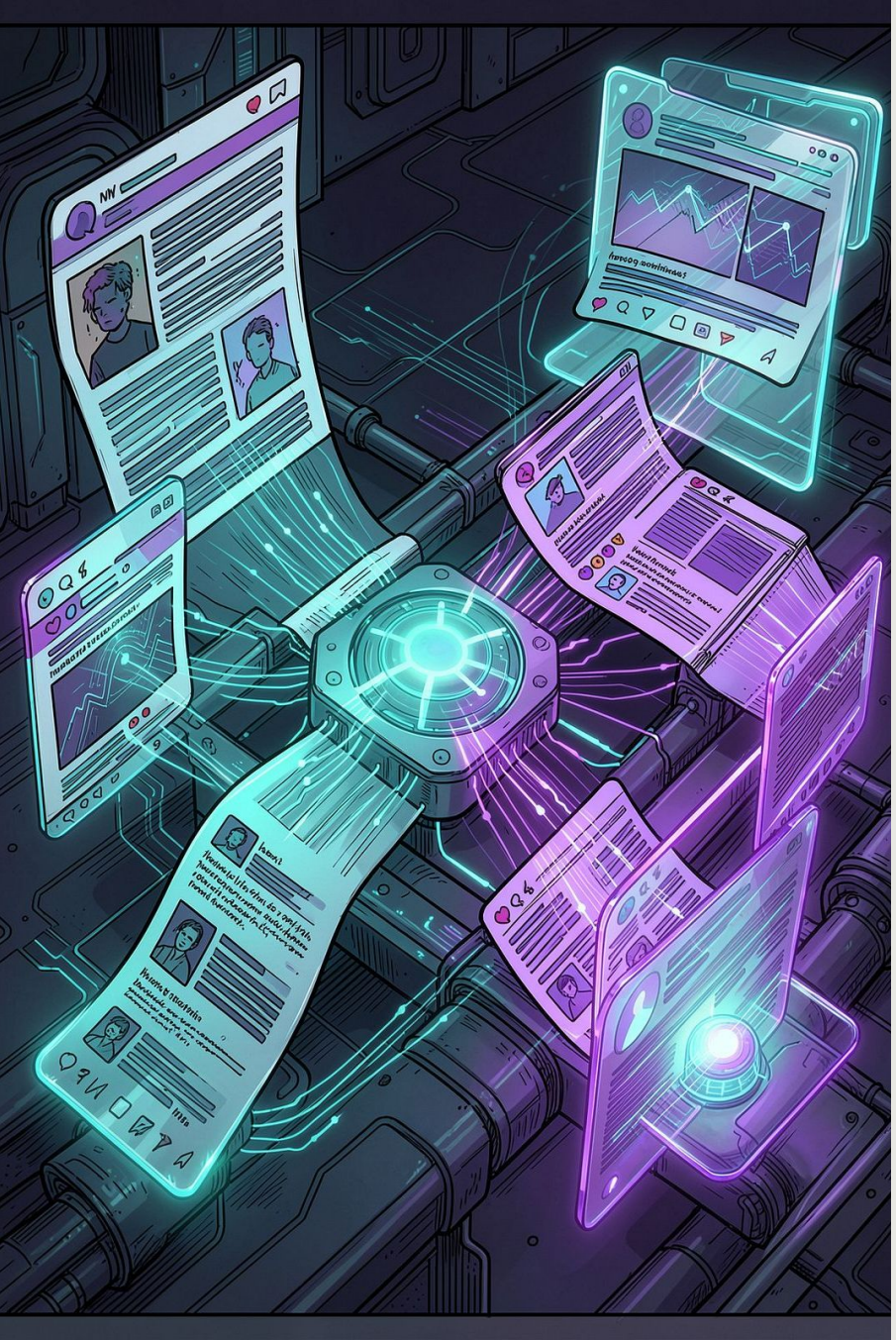


NEWS & SOCIAL MEDIA ANALYTICS ENGINE

LARGE-SCALE DATA PROCESSING USING DATA STRUCTURES AND ALGORITHMS

Data Structures & Algorithms Project

Made with GAMMA

INTRODUCTION

Millions of news articles and social-media posts are generated daily — creating scale challenges in search, organization, filtering, and moderation.



MASSIVE CONTENT

High-volume streams from news sites & social APIs.



STORAGE & SEARCH

Efficient indexing and retrieval for millions of records.



SIMILARITY DETECTION

Detect duplicates and near-duplicates at scale.



CLUSTERING

Group related stories into coherent topics.



RANKING & MODERATION

Prioritize urgent posts and assign moderators efficiently.

Visual pipeline: Massive Content → Storage → Searching → Similarity Detection → Clustering → Ranking → Moderation

Made with GAMMA

PROBLEM STATEMENT

DUPLICATE CONTENT

Identify identical or near-duplicate articles to reduce redundancy.

RELATED STORY DETECTION

Connect reports about the same event across sources and formats.

CONTENT RANKING

Prioritize posts by relevance, engagement, and urgency.

MODERATOR ASSIGNMENT

Distribute moderation tasks efficiently to reduce latency.

SCALABILITY

Process millions of records with bounded resource use.

PROCESSING SPEED

Low-latency analysis through parallel and algorithmic optimizations.

Made with GAMMA

PROPOSED SOLUTION



Key DSA at each stage: Hash tables (duplicate detection), Graphs & traversal (related stories), Clustering algorithms (topic grouping), Priority queues/heaps (ranking), and Load-balancing/parallel algorithms (moderation assignment).

Made with GAMMA

ABSTRACT

"The News & Social Media Analytics Engine is a large-scale data processing platform designed to collect, organize, and analyze news articles and social media posts efficiently. The system uses Data Structures and Algorithms (DSA) to handle millions of content records, detect duplicate or highly similar articles, group related stories into meaningful clusters, and prioritize posts for moderation. Data structures such as hash tables, graphs, priority queues, heaps, and trees are used for efficient storage, searching, similarity checking, ranking, and moderator assignment. Hashing enables fast duplicate detection, while graph-based relationships and clustering algorithms help identify articles and posts discussing the same event or topic. To support large-scale processing, the engine applies parallel processing and efficient algorithmic techniques so that multiple posts can be analyzed simultaneously instead of processing them sequentially. A priority queue can rank posts based on factors such as relevance, engagement, or moderation urgency, while a load-balancing strategy distributes moderation tasks among available moderators. The system therefore demonstrates practical applications of searching, sorting, hashing, graph algorithms, clustering, priority queues, and parallel algorithms in a real-world big-data environment. By combining these DSA concepts, the proposed engine aims to reduce duplicate information, organize related news efficiently, improve moderation workflow, and provide scalable analytics for millions of posts."



HASHING

Fast duplicate detection



GRAPHS

Relationships & traversal



CLUSTERING

Topic formation



PRIORITY QUEUE & HEAP

Ranking & urgent handling



PARALLEL PROCESSING

Scale and throughput



LOAD BALANCING

Efficient moderator assignment

Made with GAMMA

SOFTWARE & HARDWARE REQUIREMENTS

SOFTWARE REQUIREMENTS

- OS: Windows 10/11 or Linux
- Language: Java (JDK 17+)
- IDE: IntelliJ IDEA / Eclipse / VS Code
- Database: MongoDB or comparable scalable DB
- Version Control: Git & GitHub
- Visualization: Grafana / custom dashboard

HARDWARE REQUIREMENTS

- CPU: Intel Core i5 / AMD Ryzen 5 or better
- RAM: 8 GB minimum (16 GB preferred)
- Storage: 256 GB SSD minimum
- Internet: Required for API data collection
- GPU: Optional — not mandatory for core DSA



CPU



RAM



STORAGE



DB

Made with GAMMA

LITERATURE REVIEW

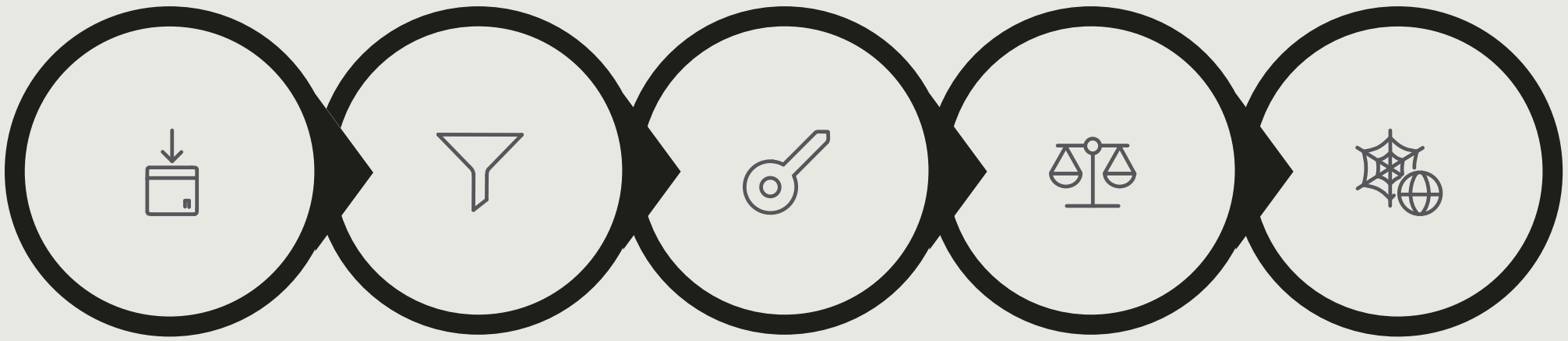
Author / Year	Research Area	Technique Used	Key Finding	Relevance
Charikar et al., 2002 (SODA / theory)	Near-duplicate detection	SimHash / locality-sensitive hashing	Efficient, scalable similarity sketching for large corpora	Duplicate detection
Leskovec, Lang & Kleinberg, 2010 (KDD/ACM)	Community detection / graphs	Graph clustering, modularity methods	Graph structure reveals topic communities at scale	Related story detection
Dhillon et al., 2003 (SIGIR)	Text clustering	Information-theoretic co-clustering / spectral methods	Robust topic grouping for document corpora	Story clustering
Ghemawat, Gobioff & Leung, 2003 (SOSP / Google)	Large-scale processing	MapReduce model	Parallel processing framework for web-scale data	Parallel algorithms
Lin et al., 2011 (WWW / arXiv)	Social media analytics	Real-time stream processing, feature extraction	Fast feature pipelines enable near-real-time analytics	Streaming & analytics

Research Gap

Most prior work focuses on individual tasks (duplicate detection, clustering, ranking, or analytics). This project integrates multiple DSA techniques into a unified large-scale analytics and moderation pipeline for news and social media.

Made with GAMMA

METHODOLOGY



Sequential steps (high level):

01

STEP 1 – DATA COLLECTION

Ingest news feeds, RSS, APIs, and social streams.

02

STEP 2 – PREPROCESSING

Normalization, tokenization, stop-word removal, metadata extraction.

03

STEP 3 – DUPLICATE DETECTION

Hashing / hash tables (SimHash / MinHash sketches).

04

STEP 4 – SIMILARITY DETECTION

Cosine / Jaccard / embedding-based similarity.

05

STEP 5 – STORY CLUSTERING

Graph construction + community detection / clustering algorithms.

06

STEP 6 – CONTENT RANKING

Priority queues / heaps to determine moderation urgency.

07

STEP 7 – MODERATOR ASSIGNMENT

Load balancer distributes tasks to available moderators.

08

STEP 8 – PARALLEL PROCESSING

MapReduce / multi-threading to improve throughput.

09

STEP 9 – ANALYTICS

Dashboards, trends, alerts, and moderation metrics.

Made with GAMMA

SYSTEM WORKFLOW



PARALLEL BRANCHES

- Worker pools process ingestion, hashing, similarity in parallel
- Graph construction runs distributed community detection
- Ranking uses distributed heaps for top-k selection
- Load balancer maintains moderator availability and latency targets

Design emphasizes non-sequential execution: concurrent stages reduce end-to-end latency on large corpora.

Made with GAMMA